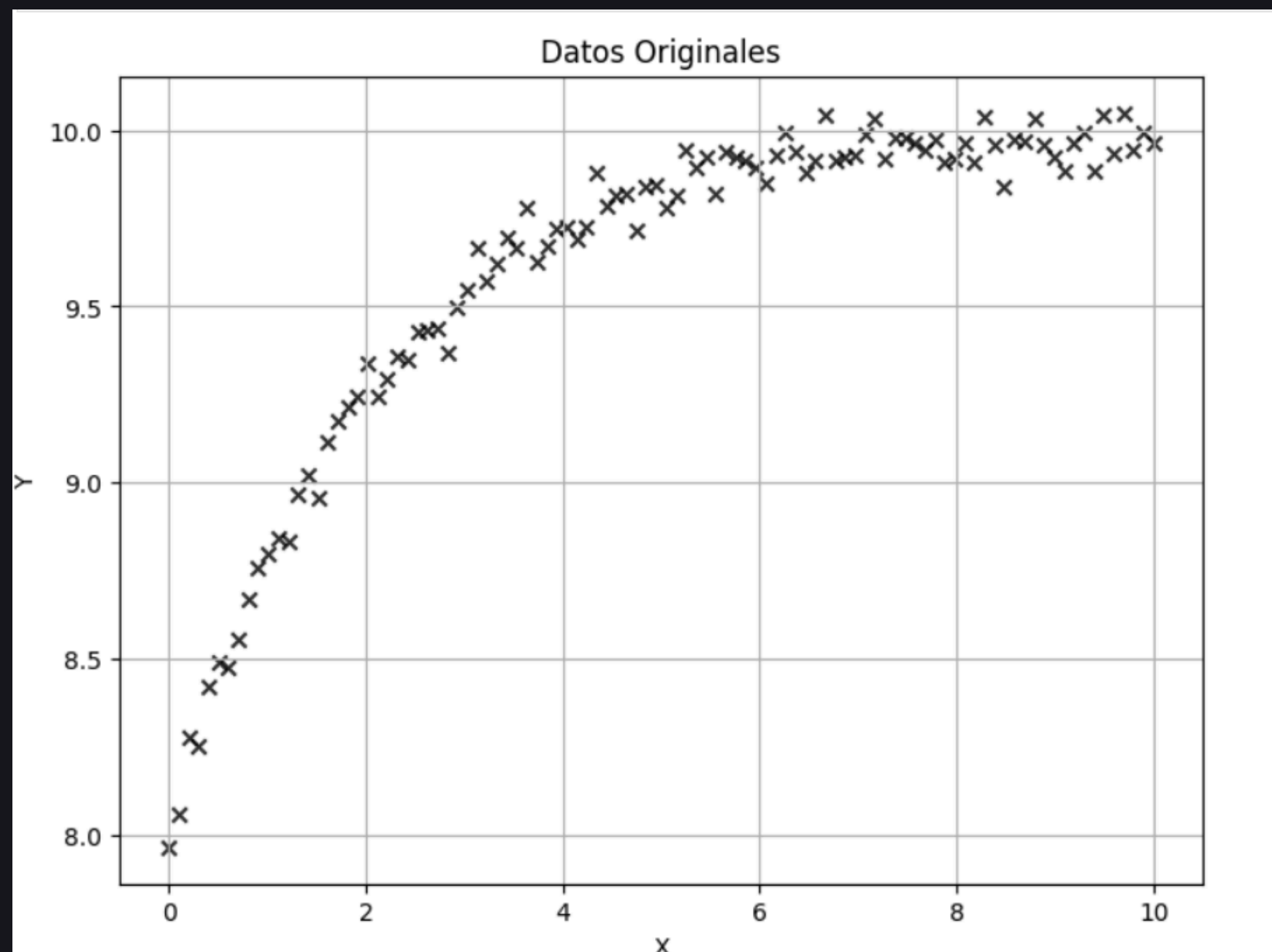


Informe 1: Interpolación y Regresión Numérica.

Interpolación de Lagrange, Interpolación mediante Splines Cúbicos, Ajustes por Regresión Lineal y No Lineal.

Introducción



DATOS GRUPO 2

1. Desarrollar una función en Python que determine el polinomio de interpolación de Lagrange que se ajuste a los datos entregados.
2. Desarrollar una función en Python que determine los **polinomios de interpolación mediante splines cúbicos**.
3. Ajustar los datos entregados mediante **regresión lineal a polinomios** de grado 1, 2, 3 y 4, y a la **función potencial** haciendo una transformación de los datos. $y = a_0x^{a_1}$
4. Ajustar los datos entregados mediante **regresión no lineal** a las siguientes funciones:

$$f(x) = a_0 - a_1e^{-a_2x}$$

$$f(x) = a_0x - a_1e^{-a_2x}$$

$$f(x) = a_0x^2 - a_1e^{-a_2x}$$

INTERPOLACIÓN DE LAGRANGE.

Interpolación de Lagrange.

Dado un conjunto de $n+1$ puntos distintos (x_0, y_0) , $(x_1, y_1), \dots, (x_n, y_n)$, el **polinomio interpolador de Lagrange** se define como:

$$f_n(x) = \sum_{i=0}^n f(x_i) L_i(x),$$

dónde los polinomios base de Lagrange $L_i(x)$ están dados por:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}.$$

```
# Función para calcular los coeficientes de Lagrange  $L_i(x)$ .
def lagrange_basis(x, x_data, i):
    n = len(x_data)
    L_i = 1
    for j in range(n):
        if j != i:
            L_i *= (x - x_data[j]) / (x_data[i] - x_data[j])
    return L_i
```

```
# Función para calcular el polinomio de interpolación de Lagrange.
def polinomio_lagrange(x, x_data, y_data):
    n = len(x_data)
    resultado = 0
    for i in range(n):
        resultado += y_data[i] * lagrange_basis(x, x_data, i)
    return resultado
```

```
x_vals = np.linspace(0, 10, 100)
y_vals = np.array([polinomio_lagrange(x, x_data, y_data) for x in x_vals])
```


Interpolación de Lagrange.

Ventajas:

- Fácil de entender e implementar.
- No requiere resolver sistemas de ecuaciones.
- Funciona con cualquier conjunto de puntos, sin necesidad de equiespaciado.

Limitaciones:

- Ineficiente para grandes conjuntos de datos.
- Fenómeno de Runge.
- Alto costo computacional.

```
# Función para calcular los coeficientes de Lagrange  $L_i(x)$ .
def lagrange_basis(x, x_data, i):
    n = len(x_data)
    L_i = 1
    for j in range(n):
        if j != i:
            L_i *= (x - x_data[j]) / (x_data[i] - x_data[j])
    return L_i
```

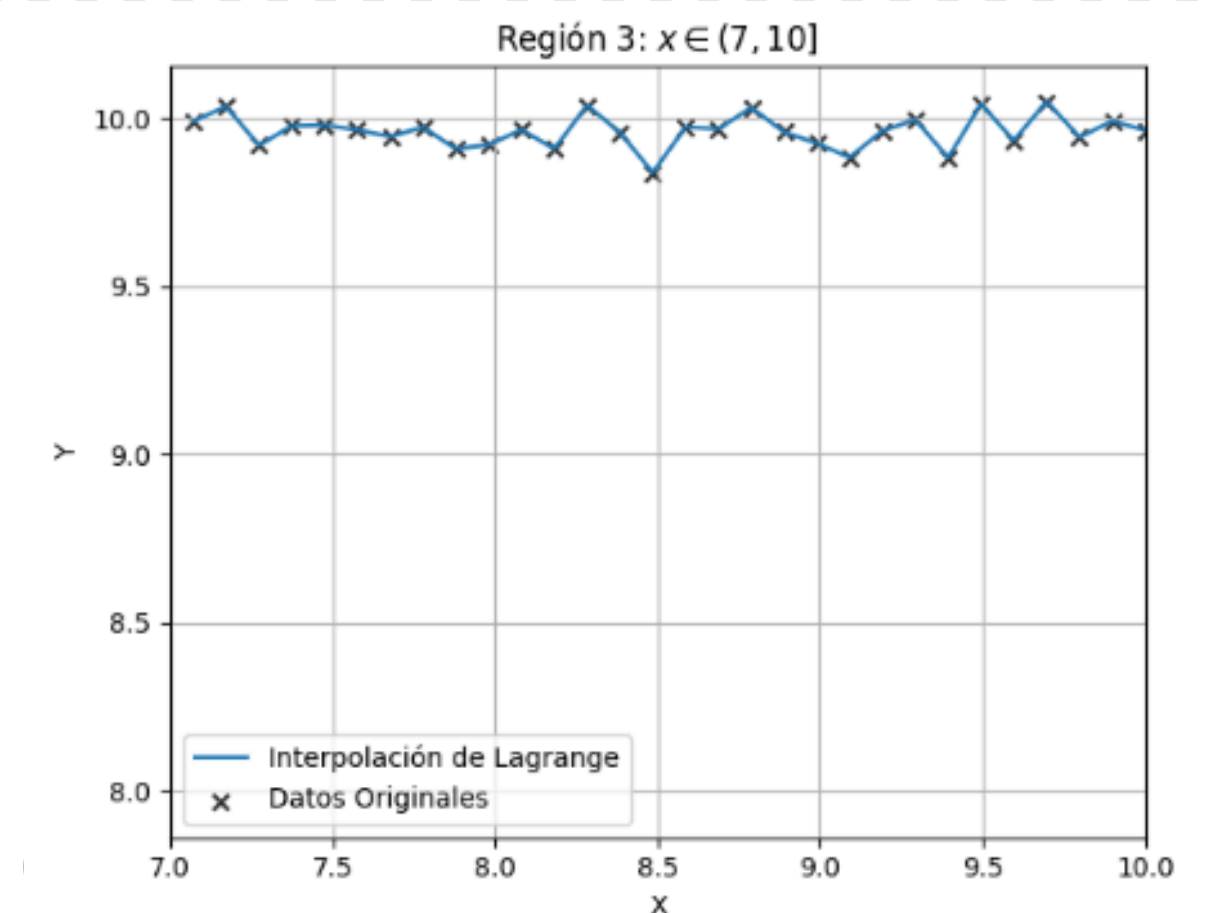
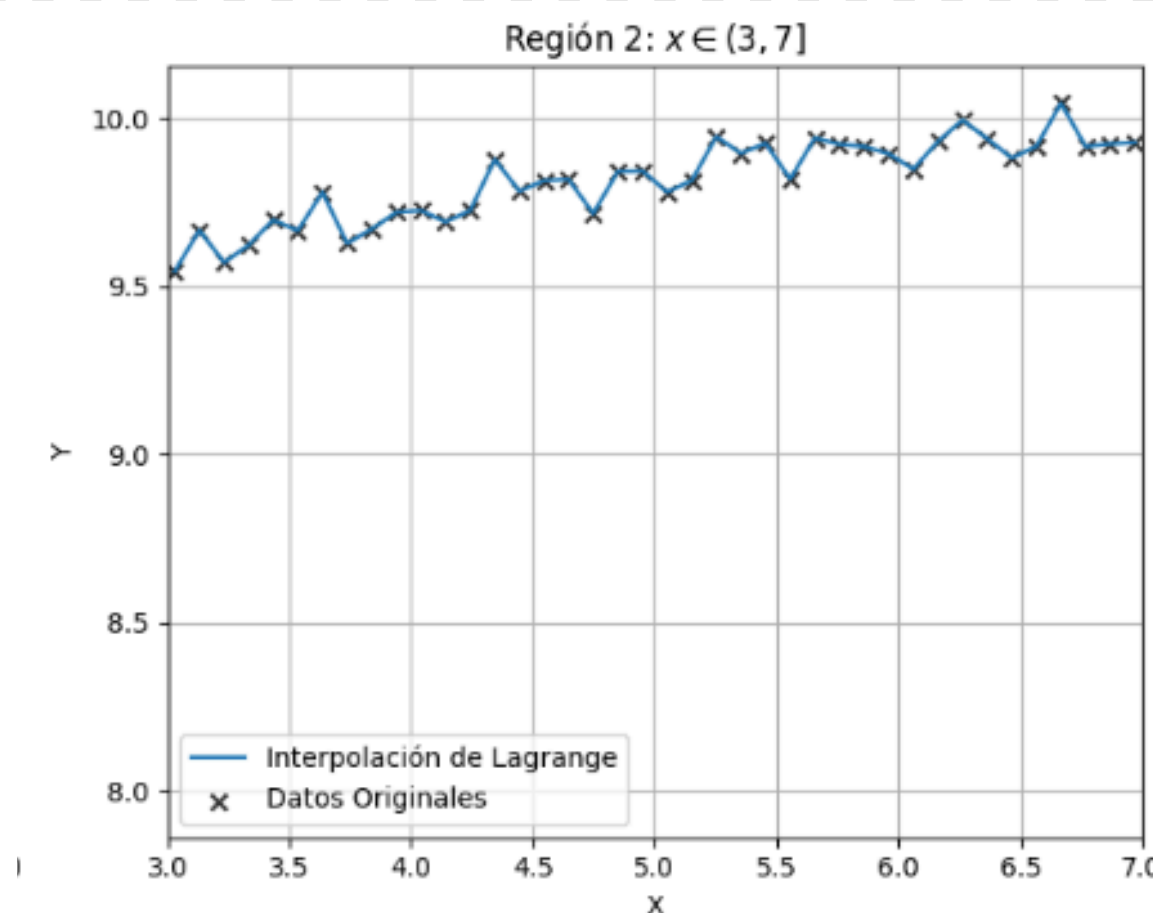
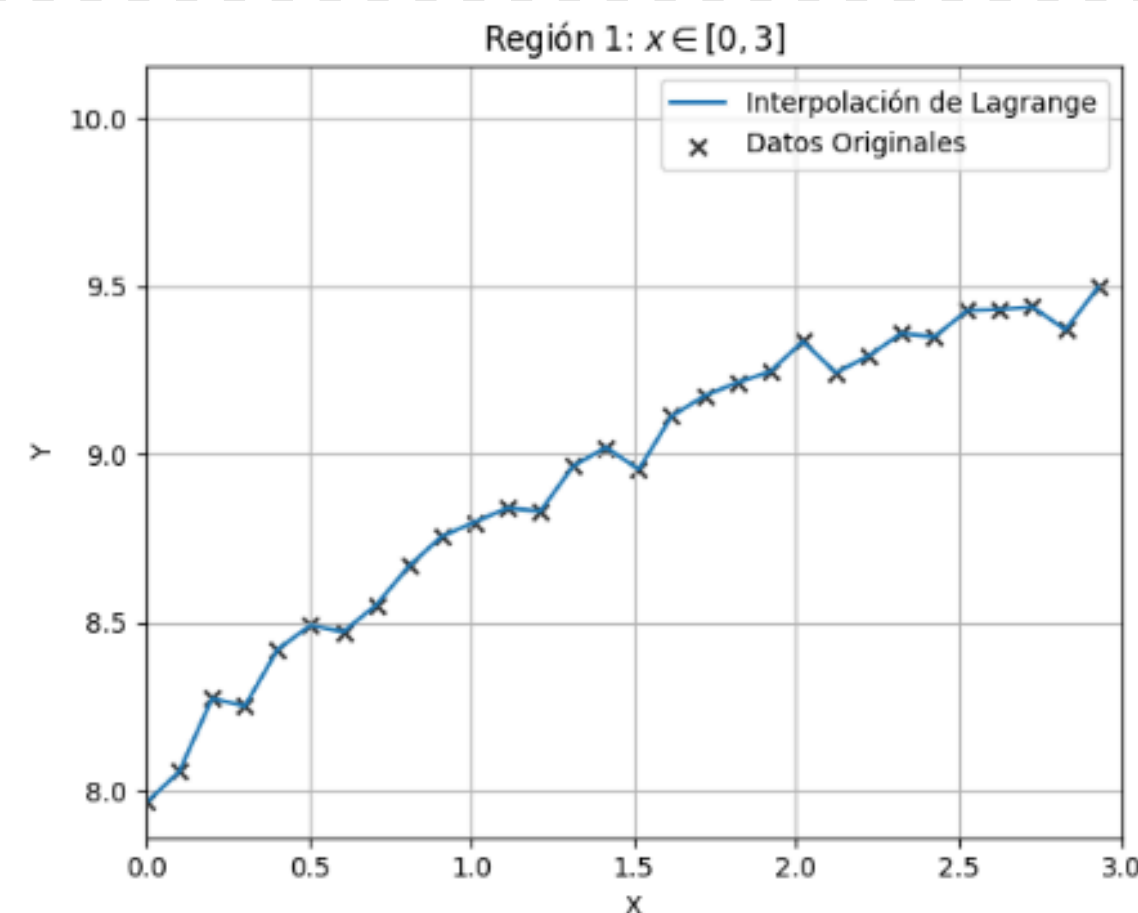
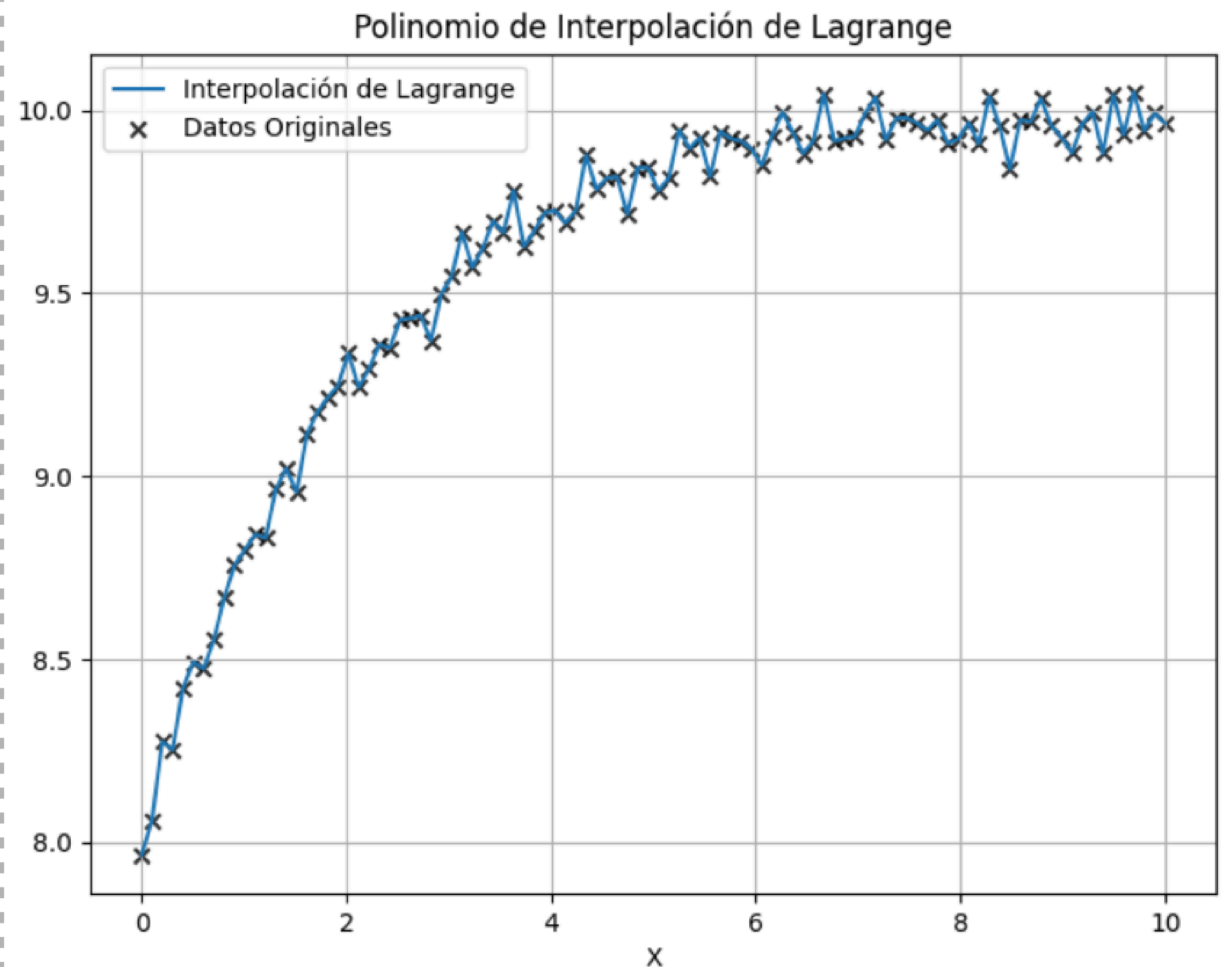
```
# Función para calcular el polinomio de interpolación de Lagrange.
def polinomio_lagrange(x, x_data, y_data):
    n = len(x_data)
    resultado = 0
    for i in range(n):
        resultado += y_data[i] * lagrange_basis(x, x_data, i)
    return resultado
```

```
x_vals = np.linspace(0, 10, 100)
y_vals = np.array([polinomio_lagrange(x, x_data, y_data) for x in x_vals])
```

INTERPOLACIÓN DE LAGRANGE.

Resultados.

- Precisión: El polinomio pasa por todos los puntos, asegurando una interpolación exacta.
- Fenómeno de Runge: Oscilaciones en los extremos, especialmente al aumentar el número de puntos.
- Limitaciones: Inestabilidad numérica con muchos nodos.



INTERPOLACIÓN MEDIANTE SPLINES CÚBICOS.

Interpolación mediante Splines Cúbicos.

- Ajusta polinomios de grado 3 en cada subintervalo.
- Ventaja: Evita oscilaciones e inestabilidad numérica.
- Comparación con Lagrange: Usa varios polinomios locales en lugar de uno de alto grado.
- Propiedad clave: Garantiza continuidad en la primera y segunda derivada.

```
# Construcción del spline cúbico.
a, b, c, d, x_knots = spline_cubico(x_data, y_data)

x_vals = np.linspace(np.min(x_data), np.max(x_data), 200)
y_vals = evaluar_spline(x_vals, x_knots, a, b, c, d)
```

```
# Construcción del spline cúbico natural. Devuelve los coeficientes de los polinomios por tramo.
def spline_cubico(x_data, y_data):
    n = len(x_data) - 1 # Número de intervalos
    h = np.diff(x_data) # Diferencia entre puntos consecutivos
    b = np.diff(y_data) / h # Diferencias divididas (pendientes entre puntos)

    # Matriz tridiagonal y vector de términos independientes
    A = np.zeros((n + 1, n + 1))
    b_vec = np.zeros(n + 1)

    # Condiciones de frontera naturales
    A[0, 0] = 1
    A[n, n] = 1

    for i in range(1, n):
        A[i, i - 1] = h[i - 1]
        A[i, i] = 2 * (h[i - 1] + h[i])
        A[i, i + 1] = h[i]
        b_vec[i] = 6 * (b[i] - b[i - 1])

    # Resolviendo el sistema de ecuaciones tridiagonal para obtener la segunda derivada (c coeficiente)
    c = np.linalg.solve(A, b_vec)

    # Calcular coeficientes a, b, c, d para cada subintervalo
    a = y_data[-1]
    d = np.diff(c) / (6 * h)
    b = b - h * (2 * c[:-1] + c[1:]) / 6

    return a, b, c[:-1]/2, d, x_data

# Evaluación del spline cúbico en un conjunto de puntos x.
def evaluar_spline(x, x_data, a, b, c, d):
    n = len(x_data) - 1
    y_vals = np.zeros_like(x)

    for i in range(n):
        indices = (x >= x_data[i]) & (x <= x_data[i + 1])
        dx = x[indices] - x_data[i]
        y_vals[indices] = a[i] + b[i] * dx + c[i] * dx**2 + d[i] * dx**3

    return y_vals
```

INTERPOLACIÓN MEDIANTE SPLINES CÚBICOS.

Definición de Spline Cúbico.

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3, \quad \text{para } x \in [x_i, x_{i+1}].$$

Construcción del sistema de ecuaciones.

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right),$$

$$\begin{bmatrix} 2(h_0 + h_1) & h_1 & 0 & \dots & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & \dots & 0 \\ 0 & h_2 & 2(h_2 + h_3) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & h_{n-2} \\ 0 & 0 & 0 & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ M_3 \\ \vdots \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} 6 \left(\frac{y_2 - y_1}{h_1} - \frac{y_1 - y_0}{h_0} \right) \\ 6 \left(\frac{y_3 - y_2}{h_2} - \frac{y_2 - y_1}{h_1} \right) \\ 6 \left(\frac{y_4 - y_3}{h_3} - \frac{y_3 - y_2}{h_2} \right) \\ \vdots \\ 6 \left(\frac{y_n - y_{n-1}}{h_{n-1}} - \frac{y_{n-1} - y_{n-2}}{h_{n-2}} \right) \end{bmatrix}$$

Cálculo de coeficientes.

$$\begin{cases} a_i = y_i, \\ b_i = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{6} (2M_i + M_{i+1}), \\ c_i = \frac{M_i}{2}, \\ d_i = \frac{M_{i+1} - M_i}{6h_i}. \end{cases}$$

INTERPOLACIÓN MEDIANTE SPLINES CÚBICOS.

Interpolación mediante Splines Cúbicos.

Ventajas:

- Mayor estabilidad numérica.
- Interpolación más precisa y suave.
- Eficiencia computacional.

Limitaciones:

- Mayor complejidad de implementación.
- No pasa exactamente por cada punto.

```
# Construcción del spline cúbico natural. Devuelve los coeficientes de los polinomios por tramo.
def spline_cubico(x_data, y_data):
    n = len(x_data) - 1 # Número de intervalos
    h = np.diff(x_data) # Diferencia entre puntos consecutivos
    b = np.diff(y_data) / h # Diferencias divididas (pendientes entre puntos)

    # Matriz tridiagonal y vector de términos independientes
    A = np.zeros((n + 1, n + 1))
    b_vec = np.zeros(n + 1)

    # Condiciones de frontera naturales
    A[0, 0] = 1
    A[n, n] = 1

    for i in range(1, n):
        A[i, i - 1] = h[i - 1]
        A[i, i] = 2 * (h[i - 1] + h[i])
        A[i, i + 1] = h[i]
        b_vec[i] = 6 * (b[i] - b[i - 1])

    # Resolviendo el sistema de ecuaciones tridiagonal para obtener la segunda derivada (c coeficiente)
    c = np.linalg.solve(A, b_vec)

    # Calcular coeficientes a, b, c, d para cada subintervalo
    a = y_data[-1]
    d = np.diff(c) / (6 * h)
    b = b - h * (2 * c[:-1] + c[1:]) / 6

    return a, b, c[:-1]/2, d, x_data

# Evaluación del spline cúbico en un conjunto de puntos x.
def evaluar_spline(x, x_data, a, b, c, d):
    n = len(x_data) - 1
    y_vals = np.zeros_like(x)

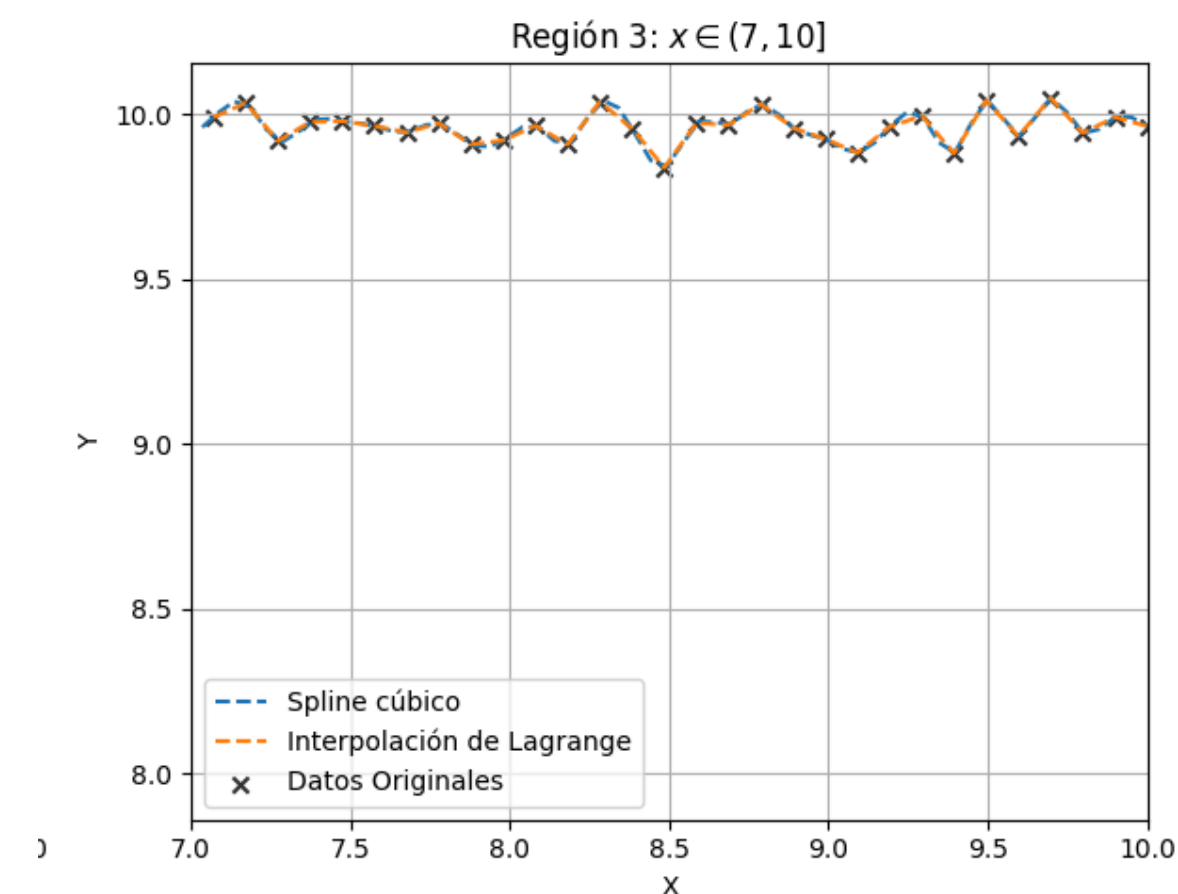
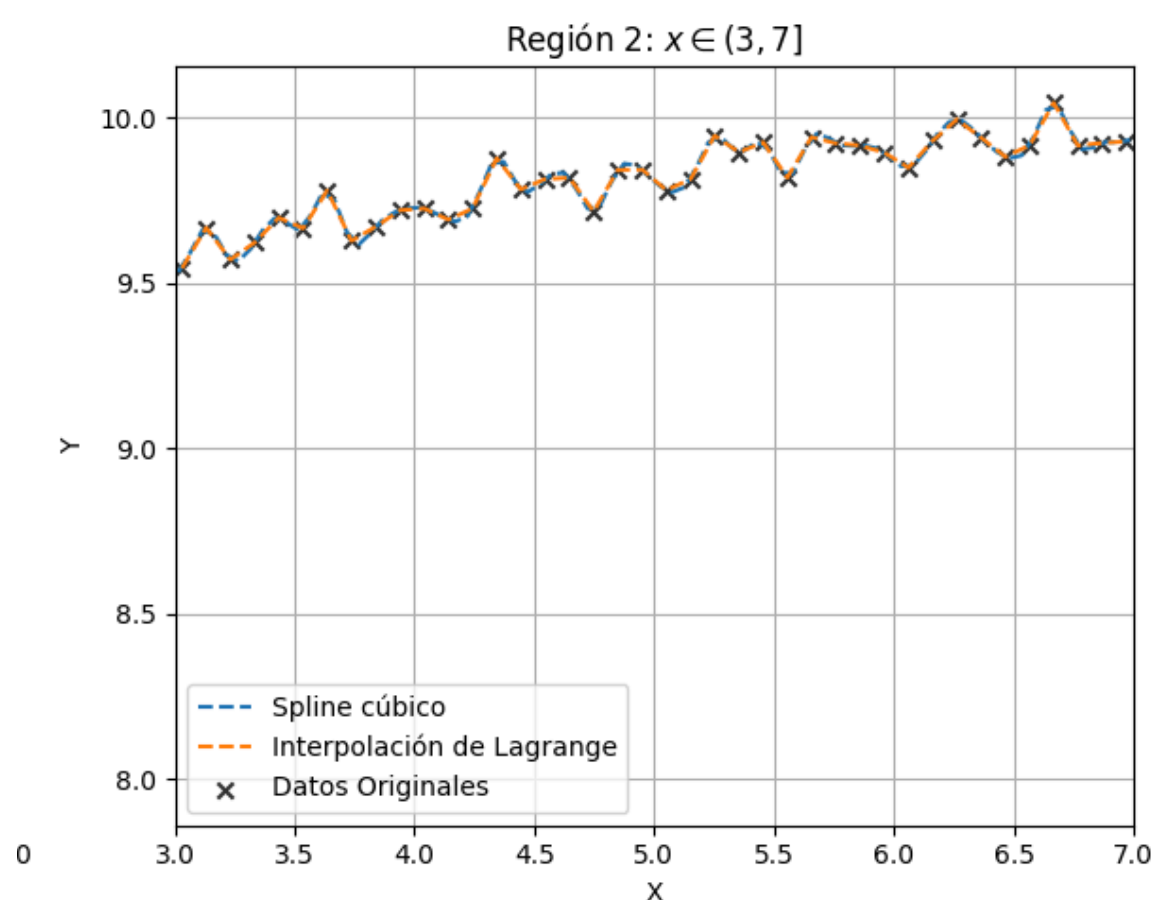
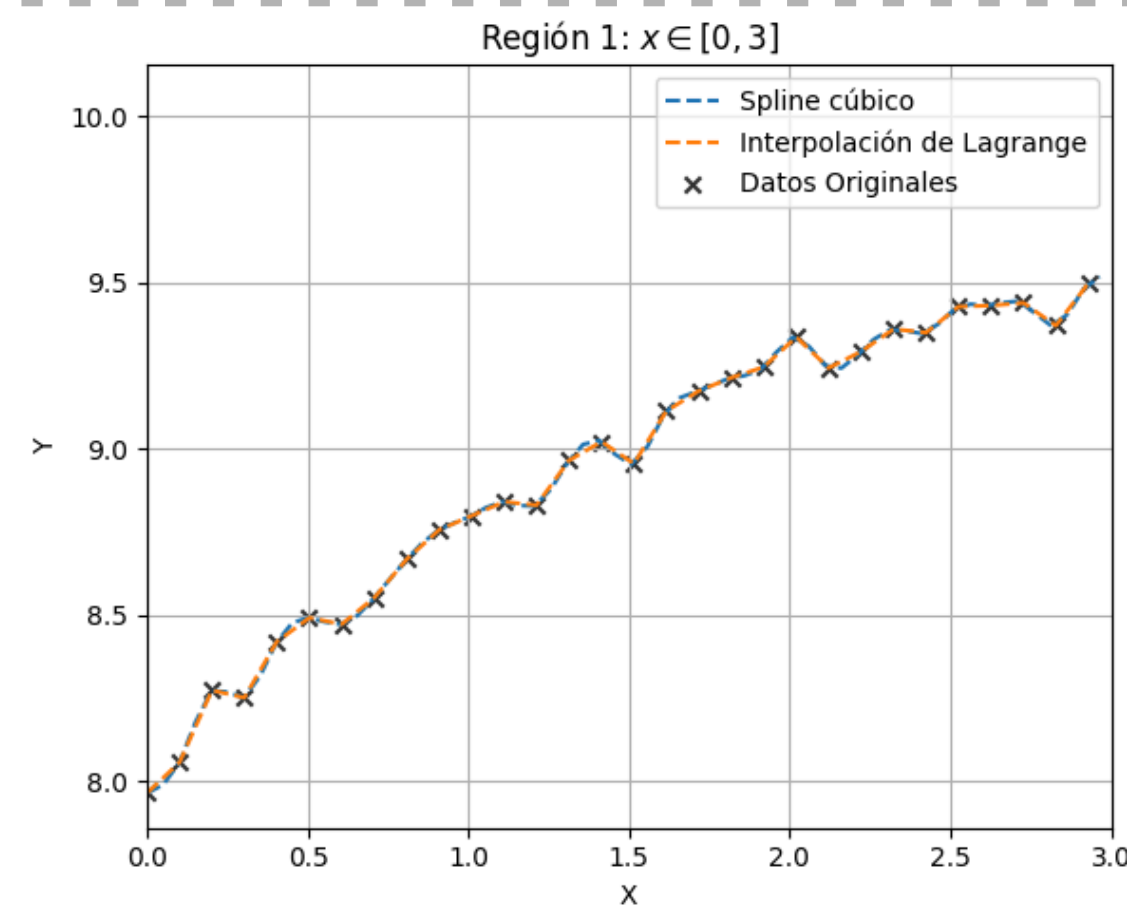
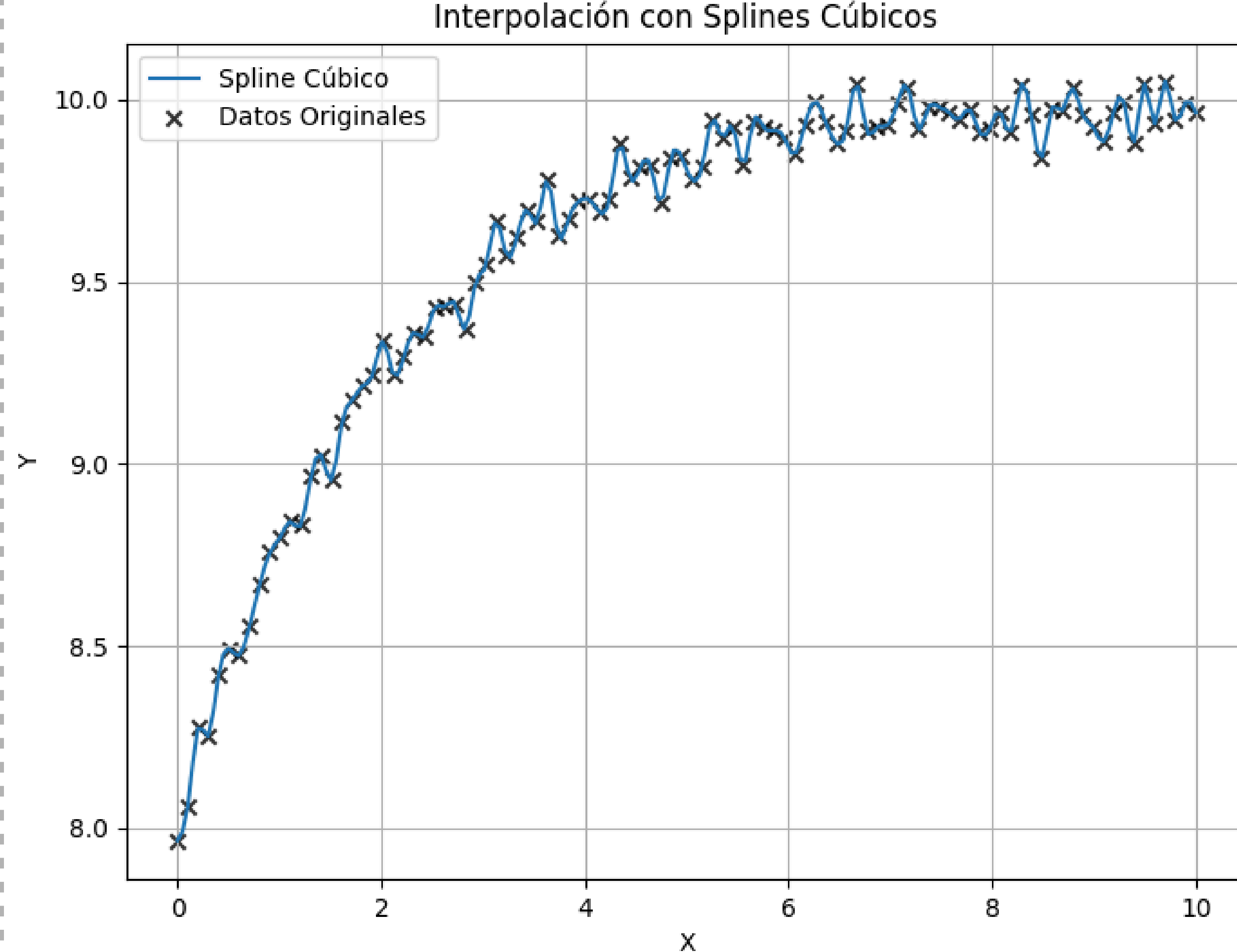
    for i in range(n):
        indices = (x >= x_data[i]) & (x <= x_data[i + 1])
        dx = x[indices] - x_data[i]
        y_vals[indices] = a[i] + b[i] * dx + c[i] * dx**2 + d[i] * dx**3

    return y_vals
```


INTERPOLACIÓN MEDIANTE SPLINES CÚBICOS.

Resultados.

- **Interpolación por tramos:** Polinomios cúbicos en cada intervalo, evitando oscilaciones bruscas.
- **Suavidad:** Garantiza continuidad en la primera y segunda derivada.
- **Estabilidad:** Más robusto y menos propenso a errores numéricos al aumentar los puntos.



Comparación de Métodos.

Criterio	Lagrange	Splines Cúbicos
Interpolación	Global (un único polinomio)	Por tramos (polinomios cúbicos)
Precisión	Buena con pocos puntos, inestable con muchos	Precisa y estable, evita oscilaciones
Eficiencia	Costosa con muchos puntos	Más eficiente (sistema tridiagonal)
Sensibilidad a cambios	Muy sensible, un cambio afecta todo el polinomio	Menos sensible, cambios locales
Aplicación recomendada	Pocos puntos, ajuste exacto en cada dato	Grandes conjuntos de datos, suavidad y estabilidad

AJUSTE POR REGRESIÓN LINEAL.

Ajuste por Regresión Polinomial.

Método de ajuste de curvas que usa un polinomio de grado n y mínimos cuadrados. Se basa en la matriz de Vandermonde y las ecuaciones normales para obtener los coeficientes del polinomio.

```
# Función para ajustar un polinomio de mediante mínimos cuadrados.
def polynomial_regression(x, y, degree):
    X = np.vander(x, degree + 1, increasing=True) # Matriz de Vandermonde.
    coeffs = np.linalg.inv(X.T @ X) @ X.T @ y # Resolver ecuación normal.
    return coeffs
```

$$\sum_{i=1}^n y_i = a_0 \sum_{i=1}^n 1 + a_1 \sum_{i=1}^n x_i + a_2 \sum_{i=1}^n x_i^2 + \cdots + a_n \sum_{i=1}^n x_i^n.$$

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 & \cdots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \cdots & \sum x_i^{n+1} \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 & \cdots & \sum x_i^{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \sum x_i^{n+2} & \cdots & \sum x_i^{2n} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \\ \vdots \\ \sum x_i^n y_i \end{bmatrix}$$

$$A = (X^T X)^{-1} X^T Y,$$

AJUSTE POR REGRESIÓN LINEAL.

Ajuste con Polinomios.

Después de ajustar un polinomio a los datos, es necesario evaluarlo en distintos puntos para visualizar su comportamiento.

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n + \varepsilon,$$

```
# Función para evaluar un polinomio en los puntos dado un conjunto de coeficientes
def evaluate_polynomial(coeffs, x_vals):
    y_vals = np.zeros_like(x_vals)
    for i, coef in enumerate(coeffs):
        y_vals += coef * (x_vals ** i) # Evaluación manual del polinomio.
    return y_vals

degrees = [1, 2, 3, 4]
fits_manual = {}

# Cálculo de los ajustes polinómicos.
for d in degrees:
    coeffs = polynomial_regression(x_data, y_data, d)
    fits_manual[d] = (coeffs, evaluate_polynomial(coeffs, x_data))

# Graficar los resultados.
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.flatten()

# Dibujo de las regresiones polinómicas.
for ax, (d, (coeffs, y_fit)) in zip(axes, fits_manual.items()):
    ax.scatter(x_data, y_data, color='black', label='Datos Originales', marker='x', alpha=0.8)
    ax.plot(x_data, y_fit, linestyle="--", label=f'Polinomio grado {d}')
    ax.set_xlabel("x")
    ax.set_ylabel("f(x)")
    ax.legend()
    ax.set_title(f"Ajuste Polinómico de Grado {d}")
    ax.grid(True)
```

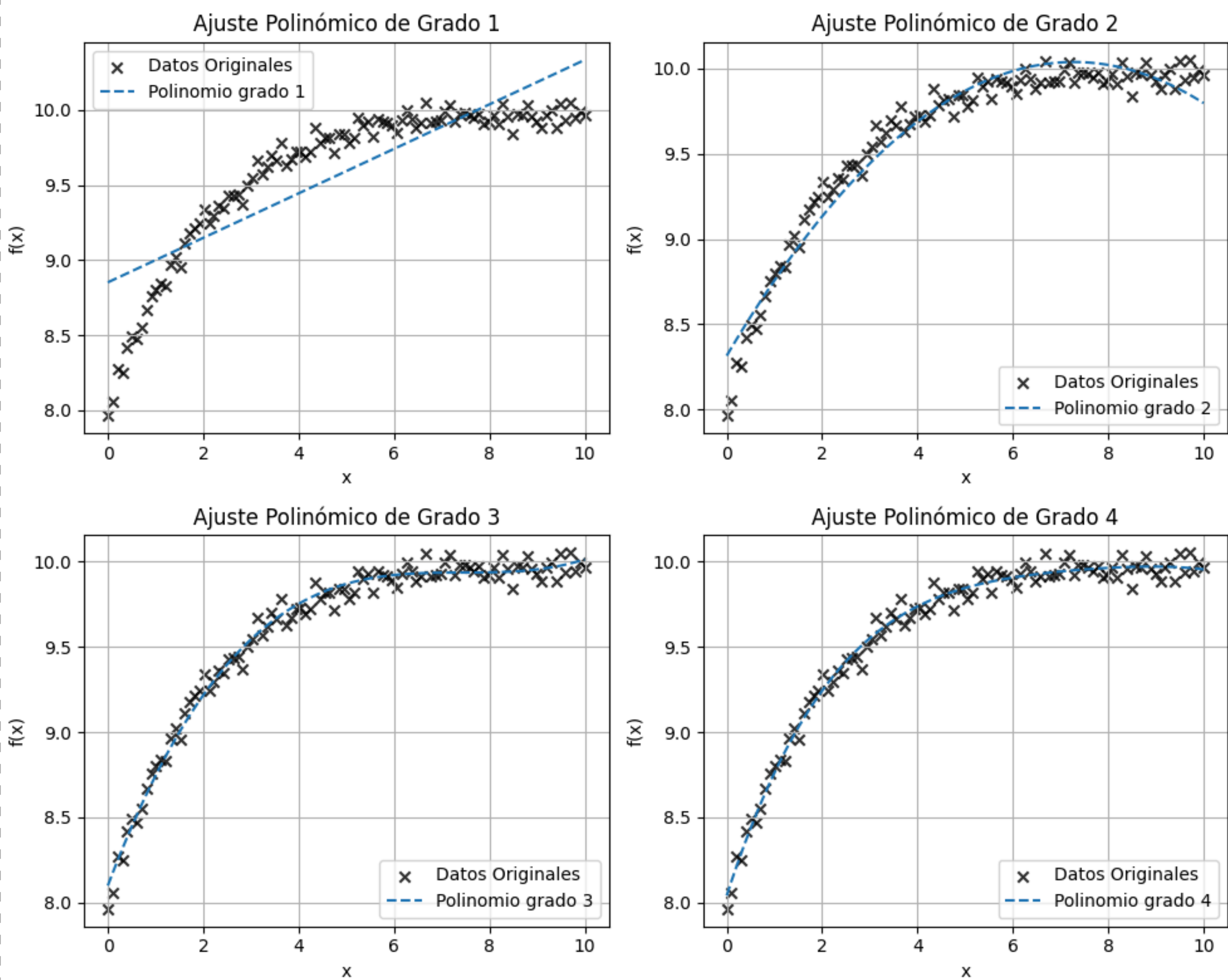

AJUSTE POR REGRESIÓN LINEAL: AJUSTE CON POLINOMIOS.

Resultados.

El ajuste mejora a medida que aumenta el grado del polinomio:

- **Grado 1:** Ajuste deficiente, no captura bien la curvatura.
- **Grado 2:** Ajuste significativamente mejor, representa mejor la tendencia.
- **Grado 3:** Ajuste preciso, captura variabilidad sin oscilaciones indeseadas.
- **Grado 4:** Mejor ajuste, pero la diferencia con grado 3 es mínima, lo que sugiere riesgo de sobreajuste.

El polinomio cúbico ofrece el mejor equilibrio entre precisión y estabilidad sin comprometer la generalización.



Grado	Ecuación del ajuste	R^2
1	$y = 8,852 + 0,148x$	0,722340
2	$y = 8,316 + 0,472x - 0,032x^2$	0,958882
3	$y = 8,100 + 0,738x - 0,099x^2 + 0,004x^3$	0,987903
4	$y = 8,042 + 0,860x - 0,154x^2 + 0,013x^3 - 0,0004x^4$	0,989690

AJUSTE POR REGRESIÓN LINEAL.

Función Potencial.

La regresión potencial modela la relación entre x e y .

Para determinar los coeficientes, se aplica una transformación logarítmica.

Esto convierte el problema en una regresión lineal.

$$y = a_0 x^{a_1}.$$

$$\ln y = \ln a_0 + a_1 \ln x$$

```
# Función potencial.
```

```
def power_law(x, a0, a1):  
    return a0 * x**a1
```

```
# Filtrar datos para evitar problemas con log(0).
```

```
valid_indices = x_data > 0  
x_nonzero = x_data[valid_indices]  
y_nonzero = y_data[valid_indices]
```

```
# Transformación logarítmica.
```

```
log_x_nonzero = np.log(x_nonzero)  
log_y_nonzero = np.log(y_nonzero)
```

```
# Ajustar una regresión lineal en la escala log-log.
```

```
coeffs = polynomial_regression(log_x_nonzero, log_y_nonzero, degree=1)
```

```
# Extracción de coeficientes.
```

```
a1_manual = coeffs[1] # Pendiente  
log_a0_manual = coeffs[0] # Intercepto en la escala log-log.  
a0_manual = np.exp(log_a0_manual) # Volver a la escala original.
```

```
# Generar predicciones con el modelo ajustado.
```

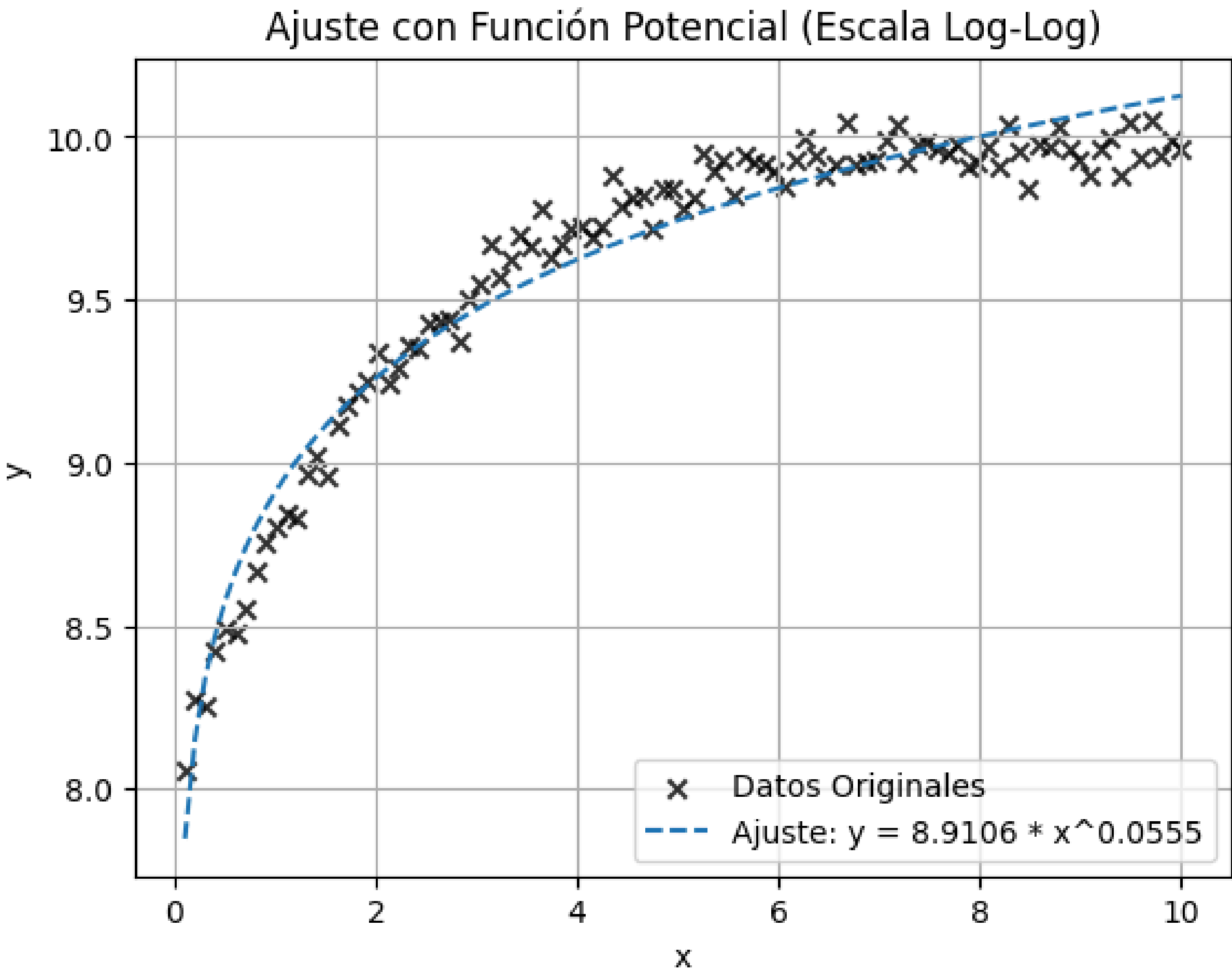
```
x_fit_nonzero = np.linspace(min(x_nonzero), max(x_nonzero), 100)  
y_fit_manual = power_law(x_fit_nonzero, a0_manual, a1_manual)
```


AJUSTE POR REGRESIÓN LINEAL: FUNCIÓN POTENCIAL.

Resultados.

La gráfica muestra el ajuste obtenido comparado con los datos originales en una escala log-log. El coeficiente de determinación indica que el modelo tiene un buen nivel de precisión y logra capturar la tendencia general de los datos.

Grado	Ecuación del ajuste	R^2
1	$y = 8,911 \cdot x^{0,056}$	0.9547



Ventajas y Limitaciones.

VENTAJAS	LIMITACIONES
Permite modelar relaciones no lineales que la regresión lineal no puede captar.	No siempre es la mejor opción para datos con tendencias más complejas.
Puede ajustarse a cualquier tipo de datos aumentando el grado del polinomio.	El exceso de flexibilidad puede llevar a sobreajuste si el grado es demasiado alto.
Con el grado adecuado, proporciona un ajuste muy preciso a los datos.	Para grados altos, puede presentar oscilaciones no deseadas (fenómeno de Runge).
Es computacionalmente eficiente para grados moderados.	El cálculo de coeficientes se vuelve inestable con polinomios de alto grado.

Ajuste por Regresión No Lineal.

Método que permite ajustar un modelo matemático a un conjunto de datos cuando la relación entre las variables no sigue una tendencia lineal. En este caso, queremos encontrar una función matemática $f(x)$ que se aproxime lo mejor posible a los datos observados.

Modelo Exponencial.

$$f(x) = a_0 - a_1 e^{-a_2 x}$$

Modelo Lineal-Exponencial.

$$f(x) = a_0 x - a_1 e^{-a_2 x}$$

Modelo Cuadrático-Exponencial.

$$f(x) = a_0 x^2 - a_1 e^{-a_2 x}$$

AJUSTE POR REGRESIÓN NO LINEAL.

Ajuste por Regresión No Lineal: Método Manual.

El método de Gauss-Newton es un enfoque iterativo basado en la linealización de la función objetivo mediante una expansión en serie de Taylor. Se minimiza la diferencia entre los datos observados y los valores predichos, resolviendo un sistema de ecuaciones en cada iteración.

El método manual (Gauss-Newton) funciona bien solo para el modelo exponencial, pero falla en los modelos 2 y 3 debido al ruido y la sensibilidad del Jacobiano.

```
def gauss_newton(x_data, y_data, modelo, jacobiano, a_init, tol=1e-6, max_iter=100):
    a_current = np.array(a_init, dtype=float) # Inicialización de parámetros.
    iter_count = 0

    while iter_count < max_iter:
        iter_count += 1

        # Evaluación el modelo en los valores actuales de los parámetros.
        y_pred = modelo(x_data, *a_current)
        residuals = y_data - y_pred

        # Cálculo del Jacobiano
        Z = jacobiano(x_data, *a_current)

        # (Z^T Z) Δa = Z^T D mediante mínimos cuadrados.
        delta_a, _, _, _ = np.linalg.lstsq(Z.T @ Z, Z.T @ residuals, rcond=None)
        a_current += delta_a

        # Criterio de convergencia.
        error = np.linalg.norm(delta_a) / np.linalg.norm(a_current)
        if error < tol:
            break

    return a_current, iter_count
```

$$\mathbf{Z}_J \Delta \mathbf{A} = \mathbf{D}, \quad \Delta \mathbf{A} = (\mathbf{Z}_J^T \mathbf{Z}_J)^{-1} \mathbf{Z}_J^T \mathbf{D}.$$

$$a_m^{(j+1)} = a_m^{(j)} + \Delta a_m.$$

AJUSTE POR REGRESIÓN NO LINEAL.

Ajuste por Regresión No Lineal: Método Automático.

El método *Curve Fit* ajusta los parámetros minimizando la diferencia entre la función ajustada y los datos experimentales de manera más eficiente que Gauss-Newton.

```
# Modelos a resolver.
modelos = {
    "Modelo Exponencial": modelo_exponencial,
    "Modelo Lineal-Exponencial": modelo_lineal_exponencial,
    "Modelo Cuadrático-Exponencial": modelo_cuadratico_exponencial
}

# Parámetros iniciales para cada modelo.
parametros_iniciales = {
    "Modelo Exponencial": [10, 5, 0.5],
    "Modelo Lineal-Exponencial": [1, 5, 0.5],
    "Modelo Cuadrático-Exponencial": [1, 5, 0.5]
}

# Diccionario para almacenar los parámetros ajustados.
parametros_ajustados = {}

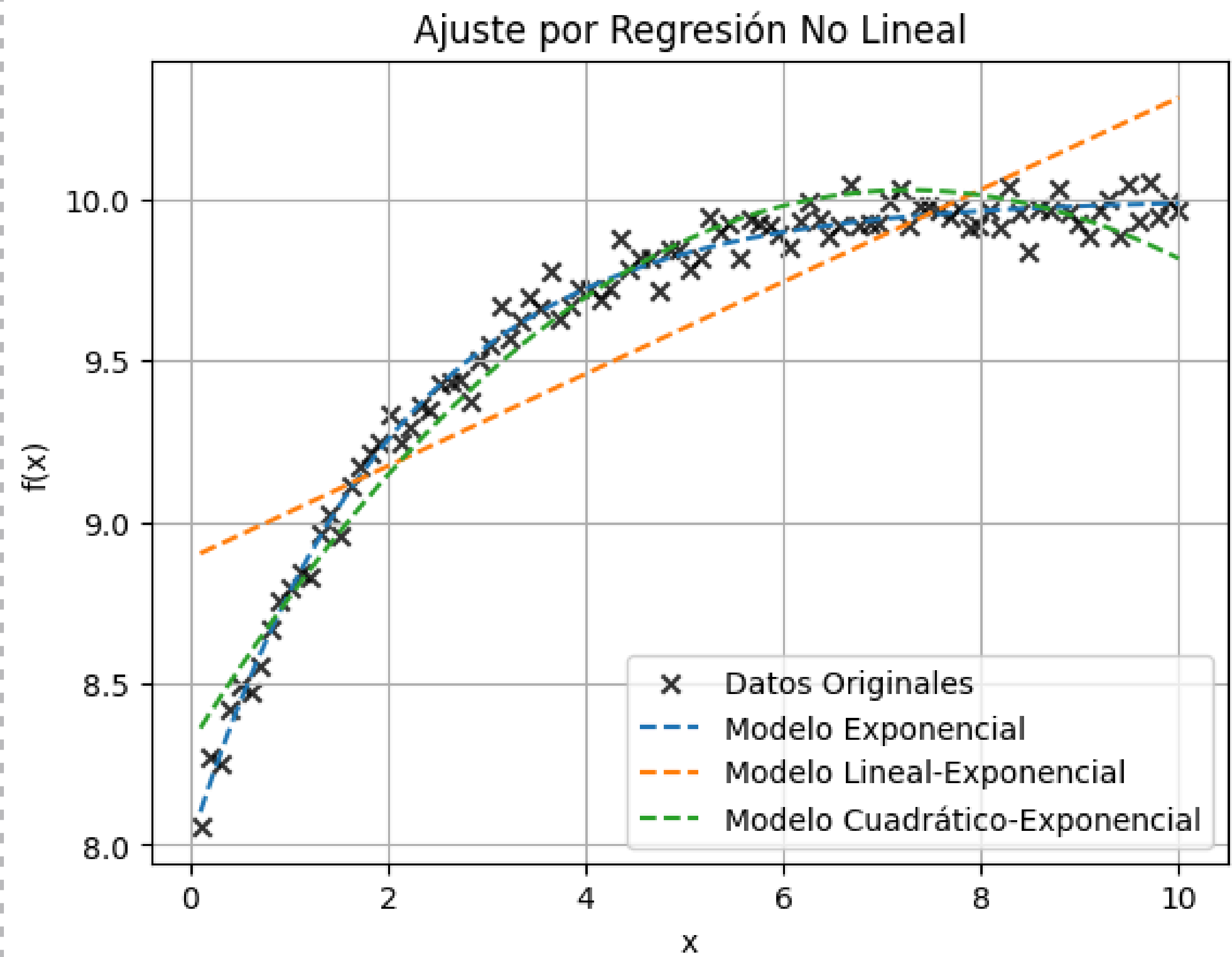
# Ajuste de los modelos.
for nombre, modelo in modelos.items():
    params_opt, _ = curve_fit(modelo, x_nonzero, y_nonzero, p0=parametros_iniciales[nombre])
    parametros_ajustados[nombre] = params_opt

# Graficar los modelos ajustados
x_fit = np.linspace(min(x_nonzero), max(x_nonzero), 200)
plt.scatter(x_nonzero, y_nonzero, color='black', label='Datos Originales', marker='x', alpha=0.8)
for i, (nombre, params) in enumerate(parametros_ajustados.items()):
    y_fit = modelos[nombre](x_fit, *params)
    plt.plot(x_fit, y_fit, linestyle='--', label=nombre, zorder=2)
```


Resultados.

- **Método Exponencial:** Mejor ajuste, captura bien la tendencia general.
- **Método Lineal-Exponencial:** Peor desempeño, se comporta casi como una regresión lineal.
- **Método Cuadrático-Exponencial:** Buen ajuste, mejor en la curvatura inicial, pero sin gran mejora en valores altos de x.

Modelo	R^2	Ecuación
Exponencial	0,991	$y = 10,0019 - 1,9965e^{-0,4918x}$
Lineal-Exponencial	0,724	$y = 0,1426x - 8,8885e^{-0,000003x}$
Cuadrático-Exponencial	0,965	$y = -0,0518x^2 - 8,3125e^{-0,0590x}$



Comparación de Modelos.

Criterio	Modelo Exponencial	Modelo Lineal-Exponencial	Modelo Cuadrático-Exponencial
Precisión	0.991 (mejor ajuste)	0.724 (peor ajuste)	0.965 (ajuste intermedio)
Captura de la tendencia general	Sí, ajusta bien a toda la curva	No, se comporta casi linealmente	Sí, pero con menor precisión que el exponencial
Eficiencia	Buena	Débil	Buena
Adecuación en valores altos de x	Buena	Débil	Intermedia
Aplicación recomendada	Modelo recomendado para describir la relación no lineal entre las variables	No recomendado, no captura bien la curvatura de los datos	Útil en casos donde se quiera enfatizar la curvatura inicial de los datos